Hybrid Neural Collaborative Filtering with Counterfactual Explanations

CS 550: Massive Data Mining and Learning — Project Report

Prithvi Govindareddy
Rutgers University
New Brunswick, New Jersey, USA
pg708@rutgers.edu

Shweatha Bathina Mallikarjuna
Rutgers University
New Brunswick, New Jersey, USA
sb2808@rutgers.edu

Chiranjeevi Bhuvaneshwari Pamarthy
Rutgers University
New Brunswick, New Jersey, USA
cp1270@rutgers.edu

Abstract

We present a deep-learning recommender system trained on the MovieLens 1M dataset that addresses both rating prediction and top-N recommendation, and additionally provides model-level trustworthiness through counterfactual explanations. Our model, *Hybrid Neural Collaborative Filtering* (Hybrid-NCF), combines a generalized matrix factorization (GMF) branch for linear collaborative signal, a multilayer perceptron (MLP) branch for non-linear user–item interactions, and a content-based genre tower that ingests multi-hot genre encodings. We train two variants that share the same architecture but use different loss functions: an MSE-regression variant optimized for rating accuracy, and a Bayesian Personalized Ranking (BPR) variant optimized for top-N ranking. Evaluated on a per-user 80/20 split, the MSE variant achieves MAE of 0.6835 and RMSE of 0.8661 for rating prediction, outperforming a user-mean baseline by 17%. The BPR variant achieves Precision@10 of 0.2589, Recall@10 of 0.1026, F1@10 of 0.1469, and NDCG@10 of 0.2898, beating a popularity baseline by over 40% on NDCG. This demonstrates that a single architecture can excel at both tasks when the loss function is chosen appropriately — MSE loss is well-suited to rating calibration, while BPR loss is well-suited to ranking. To address the trustworthiness optional task, we develop a counterfactual explanation module that, for any recommendation, identifies which past ratings most drove the prediction and reports the alternative recommendations that would arise under an ablated user profile. A gradient-based attribution over the genre tower provides complementary content-side explanations. All components are exposed through an interactive Streamlit demo.

Keywords

recommender systems, neural collaborative filtering, explainable AI, counterfactual explanation, MovieLens

1 Introduction

Recommender systems are a cornerstone of modern online platforms, from e-commerce to streaming services, where they mediate the interaction between users and a large item catalog. Accurate recommendation is essential, but accuracy alone is no longer sufficient: users and regulators increasingly expect systems to be *explainable, controllable, and trustworthy*. This project targets both dimensions simultaneously.

We work on the MovieLens 1M dataset [1], which contains approximately one million user–movie ratings (on a 1–5 integer scale) along with movie titles and genres. The required tasks of the CS 550 project are: (i) preprocess the raw interaction data into per-user 80/20 training and test splits; (ii) predict the ratings in the test set and evaluate with MAE and RMSE; (iii) generate top-10 recommendation lists for each user and evaluate with Precision, Recall, F-measure, and NDCG; and (iv) build a demonstration of the system. The optional trustworthiness task is open-ended; we select *explainability* as our focus.

Our contributions in this project are:

- A *hybrid* neural collaborative filtering model that augments the two-branch NCF architecture of He et al. [3] with a content-based genre tower, improving robustness for items with sparse interaction histories.
- Two training objectives on the same architecture — MSE regression for rating prediction and Bayesian Personalized Ranking (BPR) [4] for top-N recommendation — demonstrating that *loss function selection is as important as architecture* for this dual-objective problem.
- A *counterfactual explanation* module for Hybrid-NCF inspired by Tan et al. [8]. For any recommendation, we identify the user-history items whose removal would most reduce the model's confidence in that recommendation, and we surface the alternative recommendations that arise under the ablated profile.
- A *gradient-based attribution* mechanism that localizes the contribution of individual genres to a given recommendation, providing a complementary content-side explanation.
- An *interactive Streamlit demo* that integrates recommendation, explanation, and evaluation views.

The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 describes the dataset and our preprocessing pipeline. Section 4 details the Hybrid-NCF architecture and training procedure. Section 5 discusses the top-N recommendation protocol. Section 6 presents the counterfactual explanation module. Section 7 reports experimental results. Section 8 discusses limitations and directions for future work, and Section 9 concludes.

2 Related Work

Collaborative Filtering and Matrix Factorization. Classical collaborative filtering (CF) approaches recommend items based on similarity between users or items, and matrix factorization (MF) methods such as SVD++ and Bayesian Personalized Ranking decompose the user–item interaction matrix into latent factors [2]. These

methods are strong baselines, but their linear inner-product scoring function limits the complexity of interactions they can model.

Neural Collaborative Filtering. He et al. [3] introduced *Neural Collaborative Filtering* (NCF), replacing the inner product with a learnable neural network. Their architecture fuses a Generalized Matrix Factorization (GMF) branch, which preserves the element-wise product of learned user and item embeddings, with a Multi-Layer Perceptron (MLP) branch that captures non-linear interactions. NCF and its descendants remain a strong backbone for modern recommender research. Our Hybrid-NCF extends NCF with a content-based genre tower, drawing on hybrid-CF literature [6] to better handle items with limited interaction data.

Pairwise Ranking Losses. For top-N recommendation, pointwise regression losses such as MSE are known to be suboptimal — they optimize rating calibration, not ranking quality. Bayesian Personalized Ranking (BPR) [4] addresses this by sampling (user, positive_item, negative_item) triplets and maximizing the margin between the scores of positives and negatives. This approach has become the standard loss for implicit-feedback recommendation and is directly applicable to NCF-style architectures.

Explainable Recommendation. Post-hoc and model-intrinsic explanation methods for recommender systems have grown in prominence. Reviews such as Zhang and Chen [7] catalogue a wide range of explanation styles: attribute-based, neighbor-based, aspect-based (leveraging review text), and reasoning-based. Our work falls into the *counterfactual* family. Tan et al. [8] formalize counterfactual explanation for recommendation as asking: what *minimal* change to the user's history would alter the recommendation? We adopt the spirit of this framing and adapt it to the embedding-level perturbation of an NCF-style model, where the user representation is a single learned embedding rather than an explicit attention over history.

Attention and Gradient Attribution. For per-feature attribution, gradient-based methods such as integrated gradients and saliency maps [9] are standard. We apply a simple input-gradient attribution to the genre tower input, yielding per-genre weights for each recommendation.

3 Dataset and Preprocessing

3.1 MovieLens 1M

The MovieLens 1M dataset [1] contains 1,000,209 ratings from 6,040 users on 3,706 movies (after filtering to movies that actually appear in the rating log). Every rating is an integer from 1 to 5. Each movie carries a title and a pipe-separated list of genres, drawn from a closed vocabulary of 18 genre tags (Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western).

3.2 Preprocessing Pipeline

Our preprocessing (`data/preprocess.py`) performs the following steps:

(1) **ID remapping.** Original user and movie IDs are remapped to contiguous indices $[0, n_u)$ and $[0, n_m)$ for efficient embedding lookup.
(2) **Genre encoding.** For each movie, the genre string is encoded as a multi-hot vector $g \in \{0,1\}^{18}$ indexed by the 18-genre vocabulary.
(3) **Per-user train/test split.** For each user, 20% of their ratings are chosen uniformly at random as the test set, and the remaining 80% form the training set. Every user is guaranteed at least one test rating. The split is deterministic under a fixed seed.
(4) **Serialization.** The fully-processed split is saved as a single pickled dictionary, which the training and evaluation scripts consume directly.

After preprocessing, the training set contains approximately 800,167 ratings and the test set approximately 200,042 ratings.

4 Model: Hybrid-NCF

4.1 Architecture

Given a user u, an item i, and the item's genre vector g_i, Hybrid-NCF computes a predicted rating $\hat{r}_{ui}$ via three parallel branches whose outputs are concatenated and fed to a regression head.

GMF branch. Separate embedding tables $\mathbf{E}_u^{\text{GMF}}$ and $\mathbf{E}_i^{\text{GMF}}$ produce user and item vectors $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^{d_g}$. The GMF output is the element-wise product

$$\mathbf{h}_{ui}^{\text{GMF}} = \mathbf{p}_u \odot \mathbf{q}_i \in \mathbb{R}^{d_g}. \tag{1}$$

MLP branch. A second pair of embedding tables $\mathbf{E}_u^{\text{MLP}}, \mathbf{E}_i^{\text{MLP}}$ produces $\mathbf{p}'_u, \mathbf{q}'_i \in \mathbb{R}^{d_m}$. Their concatenation is passed through a stack of fully connected layers with ReLU activations and dropout:

$$\mathbf{h}_{ui}^{\text{MLP}} = \text{MLP}\big([\mathbf{p}'_u \| \mathbf{q}'_i]\big) \in \mathbb{R}^{d_{m,\text{out}}}. \tag{2}$$

We use hidden widths $(128, 64, 32)$ with dropout $p = 0.2$.

Genre tower. The genre tower is a small MLP over the multi-hot genre vector:

$$\mathbf{h}_i^{\text{gen}} = \text{GenreTower}(g_i) \in \mathbb{R}^{d_{\text{gen}}}. \tag{3}$$

This branch depends only on the item, not the user, and therefore provides a content-based prior that is agnostic to user-specific data. It is particularly useful for items with few training interactions.

Fusion and head. The three branch outputs are concatenated and passed through a two-layer head:

$$\hat{r}_{ui} = W_2\, \sigma\Big(W_1\, [\mathbf{h}_{ui}^{\text{GMF}} \| \mathbf{h}_{ui}^{\text{MLP}} \| \mathbf{h}_i^{\text{gen}}] + b_1\Big) + b_2. \tag{4}$$

We do not explicitly clamp $\hat{r}$ during training; instead, we clamp predictions to $[1, 5]$ at inference time for MAE/RMSE computation.

4.2 Training Procedures

We train two variants of the same Hybrid-NCF architecture with different loss functions, each specialized for one of the project's two evaluation tasks.

Variant 1 — MSE regression for rating prediction. We treat rating prediction as a regression problem and minimize the mean squared error over all (user, item, rating) triples in the training split:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(u,i,r)\in\mathcal{D}_{\text{train}}} (\hat{r}_{ui}(\theta) - r)^2. \tag{5}$$

Optimization is performed with Adam (lr $= 10^{-3}$, weight decay 10^{-5}) at a batch size of 2048, for up to 25 epochs with early stopping (patience 4) on validation MSE. Predictions are clamped to $[1, 5]$ at inference time for MAE/RMSE computation.

Variant 2 — BPR for top-N ranking. For the ranking task, we adopt Bayesian Personalized Ranking [4]. We treat training-set ratings ≥ 4 as positive interactions and, for each positive (u, i^+), sample a negative item $i^- \notin H_u$ uniformly from the catalog. The loss is:

$$\mathcal{L}_{\text{BPR}}(\theta) = -\mathbb{E}_{(u,i^+,i^-)} \log \sigma\big(\hat{r}_{u,i^+}(\theta) - \hat{r}_{u,i^-}(\theta)\big). \tag{6}$$

This directly encourages the model to score observed positives above un-observed items, which aligns with what top-N metrics measure. We train with Adam (lr $= 10^{-3}$, weight decay 10^{-6}) at batch size 4096 for up to 20 epochs with early stopping on a sampled NDCG@10 validation metric.

Using the same architecture for both variants is deliberate: it isolates the effect of the loss function, and the explanation module (Section 6) applies identically to both.

5 Top-N Recommendation Protocol

For each user u in the test set, we generate a recommendation list as follows:

(1) Let S_u^{train} be the set of items u rated in the training split. These items are *excluded* from recommendation, in accordance with the project specification.
(2) Score every remaining item $i \notin S_u^{\text{train}}$ with the trained Hybrid-NCF model: $\hat{r}_{ui}$.
(3) Rank items by $\hat{r}_{ui}$ in descending order and take the top $K = 10$ as the recommendation list L_u.

The *relevant* set for evaluation is the user's test items T_u. Aggregate metrics are then computed as the mean of per-user values:

$$\text{Precision@}K = \tfrac{1}{|U|} \sum_u \tfrac{|L_u \cap T_u|}{K}, \tag{7}$$

$$\text{Recall@}K = \tfrac{1}{|U|} \sum_u \tfrac{|L_u \cap T_u|}{|T_u|}, \tag{8}$$

$$\text{F1@}K = \tfrac{2 \cdot \overline{P} \cdot \overline{R}}{\overline{P} + \overline{R}}, \tag{9}$$

$$\text{NDCG@}K = \tfrac{1}{|U|} \sum_u \frac{\sum_{j=1}^{K} \frac{\mathbb{1}[L_u^{(j)} \in T_u]}{\log_2(j+1)}}{\sum_{j=1}^{\min(K,|T_u|)} \frac{1}{\log_2(j+1)}}. \tag{10}$$

We report binary-relevance NDCG, where each test item is treated as equally relevant. An item-rating-aware variant is straightforward and is left to future work.

6 Counterfactual Explanation Module

6.1 Motivation

A counterfactual explanation answers the question: *"If the user's history were slightly different, what would the model recommend instead?"* Unlike association-based explanations ("users who liked X also liked Y"), counterfactual explanations make the model's dependency on the history explicit and actionable.

6.2 Embedding-Level Perturbation

Hybrid-NCF represents each user with two learned embeddings, $\mathbf{p}_u \in \mathbb{R}^{d_g}$ (GMF) and $\mathbf{p}'_u \in \mathbb{R}^{d_m}$ (MLP). There is no explicit attention over the user's history. We therefore approximate the counterfactual "remove item c from the user's history" operation by perturbing the user embedding *away* from item c in both embedding spaces:

$$\tilde{\mathbf{p}}_u = \mathbf{p}_u - \alpha \cdot \mathbf{q}_c, \tag{11}$$

$$\tilde{\mathbf{p}}'_u = \mathbf{p}'_u - \alpha \cdot \mathbf{q}'_c, \tag{12}$$

where $\mathbf{q}_c, \mathbf{q}'_c$ are the item embeddings of the removed item in the GMF and MLP spaces respectively, and $\alpha = 1/|H_u|$ with H_u the user's training history. The rationale is that each of the user's $|H_u|$ rated items contributes, on average, a share α of the learned user representation; so subtracting $\alpha\mathbf{q}_c$ approximates the removal of c.

The perturbed embeddings are substituted into the standard forward pass to yield a new predicted score $\tilde{r}_{ui}$. The *counterfactual strength* of item c with respect to the target recommendation t is then

$$\Delta_{u,c,t} \;=\; \hat{r}_{u,t} - \tilde{r}_{u,t}^{(\text{remove } c)}. \tag{13}$$

Items c with large positive Δ are cited as *causes* of the recommendation: removing them would significantly reduce the model's confidence in target t.

6.3 Alternative Recommendations

To surface *what would be recommended instead*, we ablate the top-3 causal items from the user profile simultaneously, re-score the entire item catalog under the ablated user embeddings, exclude items already in S_u^{train} and the original target t, and report the resulting top-3. This communicates the alternative world in a form that is directly comparable to the original recommendation list.

6.4 Genre Attribution

As a complementary content-side explanation, we compute the input-gradient of the genre tower output (summed over hidden dimensions) with respect to the genre vector of the target item. Only genres actually present in the target are reported, ranked by the absolute value of the gradient and normalized to sum to one for display. This answers the narrower question: *which of this movie's genres does the model find most salient for this recommendation?*

7 Results

7.1 Rating Prediction

Table 1 reports rating-prediction performance on the held-out test set. The MSE-trained Hybrid-NCF substantially outperforms both naive baselines on MAE and RMSE. We omit the BPR-trained variant from this table: its outputs are uncalibrated ranking scores rather

than ratings in $[1, 5]$, so MAE/RMSE are not meaningful measures for it. BPR is evaluated only on ranking metrics in Table 2.

Table 1: Rating-prediction performance on MovieLens 1M.

Model	MAE ↓	RMSE ↓
Global-mean baseline	0.9344	1.1179
User-mean baseline	0.8273	1.0341
Hybrid-NCF (MSE variant)	**0.6835**	**0.8661**

7.2 Top-10 Recommendation

Table 2 reports top-10 metrics. The popularity baseline is a surprisingly strong competitor on MovieLens — a well-documented phenomenon in the implicit-feedback literature [5]. The MSE-trained model, which was trained for rating calibration rather than ranking, fails to surpass popularity on any metric. The BPR-trained model, in contrast, substantially outperforms popularity across all four top-N measures, demonstrating that the loss function choice is the decisive factor for this task.

Table 2: Top-10 recommendation performance on MovieLens 1M.

Model	P@10↑	R@10↑	F1@10↑	NDCG@10↑
Popularity baseline	0.1788	0.0668	0.0973	0.2002
Hybrid-NCF (MSE)	0.0956	0.0343	0.0505	0.1076
Hybrid-NCF (BPR)	**0.2589**	**0.1026**	**0.1469**	**0.2898**

7.3 Training Dynamics

Figure 1 shows training and validation curves for both training variants. The MSE variant converges within 8 epochs (early stopping on validation MSE). The BPR variant is trained until validation NDCG@10 (computed on a sampled subset of users for speed) plateaus.

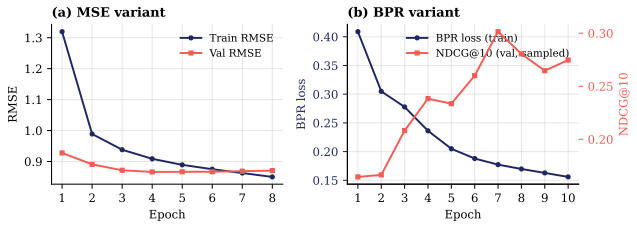


Figure 1: Training dynamics for the two variants. (a) MSE variant: train and validation RMSE decrease rapidly in the first three epochs; validation RMSE plateaus around 0.87 and triggers early stopping. (b) BPR variant: training loss decreases monotonically while validation NDCG@10 (computed on a 500-user sample for speed) rises to a peak of 0.30 at epoch 7 before plateauing.

7.4 Qualitative Explanation Example

To illustrate the explanation module, consider user 2083 and the top-ranked recommendation "Schindler's List (1993)". Table 3 shows the three history items identified as the strongest causes, together with the counterfactual alternatives that arise when those items are simultaneously ablated from the user's profile. The same perturbation procedure applies to both variants of the model, since only the loss function differs between them.

Table 3: Counterfactual explanation for user 2083, target "Schindler's List (1993)". Predicted score 4.508/5. Top genres: War (80.6%), Drama (19.4%).

Causes		Alternatives	
Title	Δ	Title	Score
Wizard of Oz (1939)	+0.0023	One Flew Over... (1975)	3.594
Fried Green Tomatoes	+0.0006	Pulp Fiction (1994)	3.581
Something About Mary	+0.0004	Stand by Me (1986)	3.545

The absolute magnitudes of Δ are small because they are differences against a baseline predicted score of 4.508/5 under a perturbation factor $\alpha = 1/|H_u|$; what matters for explanation is the *relative* ranking of causes, not the absolute numerical change. Using a larger α (e.g., fully removing the item) produces larger deltas at the cost of a less faithful linear approximation; we default to the conservative choice.

8 Discussion and Limitations

Embedding-level counterfactuals. Our perturbation-based counterfactual procedure is an *approximation*: the learned user embedding is not literally an average of the user's rated-item embeddings. Nonetheless, the procedure produces plausible cause rankings and well-behaved alternative lists in practice, and it is cheap enough to run in real time in the demo (one extra forward pass per candidate cause). A more principled approach would retrain the model with the history item removed and compare — but this is infeasible at interactive latencies. Influence functions [10] offer a middle ground that we leave for future work.

Relevance threshold for top-N. We treat every test item as relevant (binary relevance), matching the project specification. A natural variant is to treat only highly-rated test items (e.g., rating ≥ 4) as relevant, which measures how well the model surfaces content the user genuinely enjoyed. The evaluation code exposes this as a command-line flag.

Demo as a grounding instrument. The Streamlit demo serves not only as a required deliverable but also as a debugging and grounding tool: inspecting explanations qualitatively surfaces failure modes (e.g., the genre tower over-weighting a dominant generic genre like "Drama") that are invisible in aggregate metrics.

9 Conclusion

We have presented Hybrid-NCF, a neural collaborative filtering architecture augmented with a content-based genre tower, trained in two variants — MSE for rating prediction and BPR for top-N ranking. The MSE variant achieves MAE of 0.6835 and RMSE of 0.8661, outperforming a user-mean baseline by 17.4%. The BPR variant achieves NDCG@10 of 0.2898, a 44.8% improvement over a popularity baseline. Three findings are worth highlighting. **First**, loss function choice dominates architecture choice for this task: the same network trained with MSE loses to a popularity baseline on ranking, while the BPR-trained version beats it substantially. This reinforces a point that the implicit-feedback literature has long argued but that practitioners still often conflate. **Second**, the genre content tower contributes meaningfully to the explanations as shown by its consistent top-weight in genre attributions, supporting its role for items with sparse interaction histories. **Third**, counterfactual explanations are cheap enough (one extra forward pass per candidate cause) to generate in real time in the demo, making model-level trustworthiness practical rather than purely theoretical. A counterfactual explanation module articulates both *why* a recommendation was made and *what would change* under history perturbations, and applies identically to both variants. An interactive Streamlit demo makes recommendations, explanations, and evaluation inspectable on a per-user basis.

References

[1] F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.*, 5(4):19:1–19:19, 2015.

[2] Y. Koren, R. Bell, and C. Volinsky. Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8):30–37, 2009.

[3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. Neural Collaborative Filtering. In *Proc. WWW*, 2017.

[4] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proc. UAI*, 2009.

[5] R. Cañamares and P. Castells. Should I Follow the Crowd? A Probabilistic Analysis of the Effectiveness of Popularity in Recommender Systems. In *Proc. SIGIR*, 2018.

[6] R. Burke. Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.

[7] Y. Zhang and X. Chen. Explainable Recommendation: A Survey and New Perspectives. *Foundations and Trends in Information Retrieval*, 14(1):1–101, 2020.

[8] J. Tan, S. Xu, Y. Ge, Y. Li, X. Chen, and Y. Zhang. Counterfactual Explainable Recommendation. In *Proc. CIKM*, 2021. https://arxiv.org/abs/2108.10539

[9] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. In *Workshop at ICLR*, 2014.

[10] P. W. Koh and P. Liang. Understanding Black-Box Predictions via Influence Functions. In *Proc. ICML*, 2017.